

Data Structures Practical Test

Topics: Linked List, Queue, Priority Queue, Stack | Difficulty: Intermediate

Instructions: Solve all questions. Write clean, modular code and test your implementation with appropriate test cases.

Section A — Linked List

Question 1 — Rotate a Linked List

A company's pending tasks are stored in a singly linked list. Given the head of the list and an integer **k**, rotate the linked list to the **right by k positions**.

Example:

Input: $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5$, $k = 2$

Output: $4 \rightarrow 5 \rightarrow 1 \rightarrow 2 \rightarrow 3$

Requirements: Implement the solution using a linked list. Do not create a new linked list. Handle cases where k is greater than the length of the list, an empty list, and a single-node list.

Question 2 — Remove Nth Node From the End

A system stores records in a singly linked list. Given the head of the list and an integer **n**, remove the **nth node from the end** of the list.

Example:

Input: $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5$, $n = 2$

Output: $1 \rightarrow 2 \rightarrow 3 \rightarrow 5$

Requirements: Use a two-pointer approach. Do not calculate the length separately. Handle removal of the first node and invalid values of n .

Section B — Queue

Question 3 — Circular Queue

A server has a fixed-size buffer for incoming requests. The buffer can store only **N** requests at a time. Once full, new requests cannot be added until an existing request is removed.

Implement a **Circular Queue** with: `enqueue(value)`, `dequeue()`, `front()`, `is_empty()`, and `is_full()`.

Example: Queue size = 3; `enqueue(10)`, `enqueue(20)`, `enqueue(30)`, `dequeue()`, `enqueue(40)` $\rightarrow 20 \rightarrow 30 \rightarrow 40$.

Requirements: Do not use Python's built-in queue. Handle overflow/underflow and maintain front/rear correctly.

Question 4 — First Non-Repeating Character

A system receives characters one at a time. After every new character, determine the **first character that has appeared only once** so far. If none exists, return -1.

Example: Input: a, a, b, c $\rightarrow$ Output: a, -1, b, b

Requirements: Use a queue and an appropriate frequency-tracking structure. Process characters efficiently.

Section C — Priority Queue

Question 5 — Hospital Emergency Queue

A hospital receives patients with different emergency severity levels. Each patient has a Patient ID, Patient Name, Severity, and Arrival Order. A patient with a **higher severity** must be treated first. If severity is equal, the patient who arrived earlier is treated first.

Example: A(3), B(5), C(5), D(2) → treatment order: B, C, A, D.

Requirements: Implement using a priority queue. Support adding a patient, treating/removing the highest-priority patient, and viewing the next patient. Correctly handle equal priorities.

Question 6 — K Largest Numbers

You are given a continuous stream of integers and an integer **K**. Maintain the **K largest numbers** seen so far without sorting the complete collection every time.

Example: Numbers = [10, 5, 20, 8, 15, 30], K = 3 → [20, 30, 15].

Requirements: Use a heap/priority queue. Maintain only the required number of elements. Explain time and space complexity. Handle duplicates correctly.

Section D — Stack

Question 7 — Balanced Brackets

A code editor needs to verify whether brackets in a piece of code are correctly balanced. The input may contain `()`, `{}`, and `[]`. Brackets must be properly opened and closed.

Example 1: `{[()]}` → True

Example 2: `{[(())]}` → False

Requirements: Use a stack. Support all three bracket types and correctly handle nested brackets.

Question 8 — Next Greater Element

A company records daily sales numbers. For every day, find the **next day to the right with a greater sales value**. If there is no greater value, return -1.

Example: Input: [10, 5, 12, 8, 15] → Output: [12, 12, 15, 15, -1]

Requirements: Use a stack. Try to achieve **O(n)** time complexity. Do not use nested loops to compare every element. Explain the stack pattern used.